CS3630 Lab 1: RGB-D Tabletop Instance Segmentation

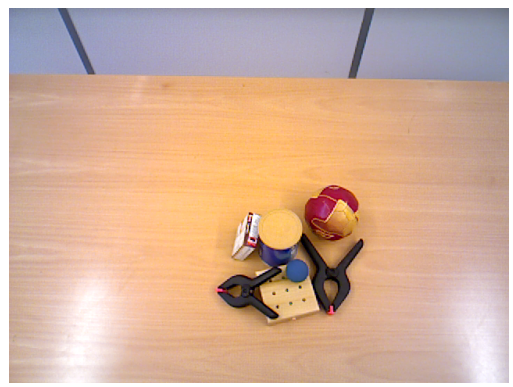
Georgia Tech, Spring 2026

Overview: The robot tabletop segmentation challenge

Imagine you are programming a robot assistant tasked with cleaning a cluttered house. To successfully manipulate objects—like picking up a mug or moving a box—the robot must first understand the scene geometry. Specifically, it needs to distinguish the supporting surface (the table) from the items resting on it and identify each distinct object as a unique instance.

In this lab, you will implement a perception pipeline to solve this tabletop instance segmentation problem using the OCID dataset. This dataset provides RGB-D images of cluttered tabletop environments with diverse household objects, backgrounds, and occlusion patterns. Your goal is to produce an instance mask where the background (table, floor, walls) is labeled as 0, and every distinct object is assigned a unique positive integer ID $(1, 2, \ldots, N)$.

(a) A service robot interacting with the environment.



(b) An RGB example from the dataset displays a cluttered tabletop scene the robot must segment.

Figure 1: The Context: Robot perception bridging raw data and interaction.

Step 1: Background Subtraction

A robust strategy for tabletop scenes is background subtraction. Since the table is a flat, dominant surface, we can model it as a plane. By detecting and removing this plane, the remaining points can be treated as foreground objects. To find this plane robustly despite the "outliers" (the objects on the table), **your task is to implement RANSAC**.

As discussed in the lecture, the RANSAC algorithm for plane fitting is an iterative process. In each iteration, the algorithm randomly samples three points – the minimum required to define a plane – and fits a candidate plane model to them. Then it counts the number of "inliers," which are points whose distance from the plane falls within a specific threshold t. This process is repeated for N iterations to ensure a high probability of finding the true table plane. Finally, the plane model with the largest set of inliers is selected as the table.

Recall the rule of thumb for determining the number of iterations N required to succeed with probability p, given an expected inlier ratio w and sample size $n = 3$:

$$N = \frac{\log(1-p)}{\log(1-w^n)}$$

This formula highlights that as the scene becomes more cluttered (lower w), you need significantly more iterations to guarantee finding the table.

Step 2: Object Segmentation & Your Custom Strategy

Once the background is removed, the robot is left with clusters of points. Our provided baseline implementation uses Connected Components Labeling (CCL) to group these clusters into unique instance IDs. CCL operates on the binary foreground mask to group spatially connected pixels into distinct regions, effectively assigning a unique label to each isolated object cluster. However, the baseline is just a starting point. While RANSAC and CCL work well for simple scenes, they are often insufficient for the scenario where objects are touching or lighting is poor.

Your goal in this step is to design and implement a more robust segmentation strategy that improves upon the baseline. You are free to explore any approach that leverages the available modalities — RGB, depth, surface normals, or a combination of both.

Evaluation

Datasets: Easy vs. hard splits

Your perception pipeline will be benchmarked against two distinct subsets of the OCID dataset, representing different levels of robotic difficulty:

- **The Easy Dataset:** This subset features simple geometric shapes and a limited number of objects per scene. Lighting is consistent, and objects are well-separated, making it ideal for verifying your RANSAC baseline.
- **The Hard Dataset:** This subset is designed to test the limits of your custom solution. It contains a significantly higher object density, varied lighting conditions, and objects with complex textures.

The lab includes a representative sample of these datasets for local testing and visualization. You should use these files to estimate your performance and debug your code. However, please note that when you submit to Gradescope, your solution will be evaluated against the full, hidden dataset. Ensure your custom method is robust and generalized rather than tuned specifically to the sample images provided.

Calculating the metrics

We evaluate your submission using Panoptic Quality (PQ). This metric combines your ability to detect objects (Recognition Quality) with your ability to precisely outline them (Segmentation Quality).

Before calculating the score, we must match your predicted masks to the Ground Truth (GT). Note that in the OCID dataset, GT labels 0, 1, and 2 are reserved for background (floor, table, etc.).

Therefore, any predicted pixel with an ID > 0 is considered a predicted object, and any GT pixel with an ID > 2 is considered a ground truth object.

We compute the Intersection over Union (IoU) for every possible pair of predicted instance and GT instance. We then perform Hungarian matching to maximize the total IoU of the assignment. A match is considered a True Positive (TP) only if IoU ≥ 0.5. Any unmatched predicted instances count as False Positives (FP), and unmatched ground truth instances count as False Negatives (FN).

How do we calculating the Score

The final PQ score is the product of two sub-metrics:

1. Segmentation Quality (SQ): This measures how precise your masks are. It is the average IoU of all matched pairs:

$$\mathrm{SQ} = \frac{1}{|\mathrm{TP}|} \sum_{(p,g)\in \mathrm{TP}} \mathrm{IoU}(p, g)$$

2. Recognition Quality (RQ): This measures your detection accuracy (i.e. did you find the correct number of objects?). It is essentially the F1-score:

$$\mathrm{RQ} = \frac{|\mathrm{TP}|}{|\mathrm{TP}| + 0.5|\mathrm{FP}| + 0.5|\mathrm{FN}|}$$

The Final Score:

$$\mathrm{PQ} = \mathrm{SQ} \times \mathrm{RQ}$$

Understanding this metric is crucial for debugging. If your PQ is low, check if you are over-segmenting objects (high FP, lowers RQ) or if your masks are messy and don't align well with boundaries (low IoU, lowers SQ).

Getting Started

To begin, please read the `README.md` file in the repository. It contains specific installation commands, the grading rubric, and submission details.

Your implementation tasks are centered on `utils/ransac_plane.py`, where you will implement the `fit_plane_from_3pts` function and the main `ransac_plane` loop. Once the baseline is functional, you will edit `solution.py` to implement a custom method. You are encouraged to improve upon the baseline by exploring all your methods to outperform the baseline on the "hard" dataset split. Good luck on the lab!